

Paso 11 - Validaciones | Solucionario

Las cinco soluciones del paso 11. Todas comparten el mismo `Validador.java`, que va creciendo: aquí está entero, con los tres métodos que agregan los ejercicios.

Úsalo después de intentarlo. Cada solución viene con la salida real del programa: si la tuya coincide, está bien aunque el código sea distinto. Los `.java` de esta carpeta son los mismos que se muestran abajo, listos para ejecutar: `cd soluciones/paso11` y `javac *.java`.

B11.1 - leerSiNo

Un `leer...` más, escrito **encima** de `leerTexto`: no vuelve a tocar el `Scanner`.

▼ Ver la solución (Validador.java, B11_1_LeerSiNo.java)

Validador.java

```

import java.util.Scanner;

/*
 * VALIDADOR - el del paso 11, con los tres metodos de la hoja agregados.
 *
 * es... / esta... PREGUNTAN: devuelven true o false.
 * leer... INSISTEN: piden hasta que el dato sirva.
 */
public class Validador {

    // ----- los que PREGUNTAN -----

    public static boolean estaEnRango(double valor, double min, double max) {
        return valor >= min && valor <= max;
    }

    public static boolean esCorreoDuoc(String correo) {
        String dominio = "@duocuc.cl";
        return correo.endsWith(dominio)
            && correo.length() > dominio.length()
            && !correo.contains(" ");
    }

    // B11.2
    public static boolean esClaveSegura(String clave) {
        if (clave.length() < 8) {
            return false;
        }
        boolean tieneDigito = false;
        boolean tieneMayuscula = false;
        for (int i = 0; i < clave.length(); i++) {
            char c = clave.charAt(i);
            if (Character.isDigit(c)) {
                tieneDigito = true;
            }
            if (Character.isUpperCase(c)) {
                tieneMayuscula = true;
            }
        }
        return tieneDigito && tieneMayuscula;
    }
}

```

```

// B11.4
public static boolean esRutValido(String rut) {
    int guion = rut.indexOf('-');
    if (guion < 7 || guion != rut.lastIndexOf('-')) {
        return false; // sin guion, muy corto, o con
dos
    }
    if (rut.length() != guion + 2) {
        return false; // despues del guion va UN
caracter
    }
    for (int i = 0; i < guion; i++) {
        if (!Character.isDigit(rut.charAt(i))) {
            return false;
        }
    }
    char dv = rut.charAt(guion + 1);
    return Character.isDigit(dv) || dv == 'k' || dv == 'K';
}

// B11.4 extra: el digito verificador de verdad (modulo 11)
public static boolean tieneDvCorrecto(String rut) {
    if (!esRutValido(rut)) {
        return false;
    }
    int guion = rut.indexOf('-');
    String cuerpo = rut.substring(0, guion);

    int suma = 0;
    int multiplicador = 2;
    for (int i = cuerpo.length() - 1; i >= 0; i--) { // de derecha a
izquierda
        suma += (cuerpo.charAt(i) - '0') * multiplicador;
        multiplicador++;
        if (multiplicador > 7) {
            multiplicador = 2;
        }
    }

    int resto = 11 - (suma % 11);
    char esperado;
    if (resto == 11) {
        esperado = '0';
    }

```

```

    } else if (resto == 10) {
        esperado = 'K';
    } else {
        esperado = (char) ('0' + resto);
    }

    return Character.toUpperCase(rut.charAt(guion + 1)) == esperado;
}

// ----- los que INSISTEN -----

public static String leerTexto(Scanner sc, String pregunta) {
    System.out.print(pregunta);
    String texto = sc.nextLine().trim();
    while (texto.isEmpty()) {
        System.out.println("    No puede quedar vacio.");
        System.out.print(pregunta);
        texto = sc.nextLine().trim();
    }
    return texto;
}

public static int leerEntero(Scanner sc, String pregunta, int min, int max)
{
    while (true) {
        System.out.print(pregunta);
        if (!sc.hasNextInt()) {
            System.out.println("    Eso no es un numero entero.");
            sc.nextLine();
        } else {
            int valor = sc.nextInt();
            sc.nextLine();
            if (estaEnRango(valor, min, max)) {
                return valor;
            }
            System.out.println("    Tiene que ir de " + min + " a " + max +
".");
        }
    }
}

public static double leerDecimal(Scanner sc, String pregunta, double min,
double max) {

```

```

        while (true) {
            System.out.print(pregunta);
            if (!sc.hasNextDouble()) {
                System.out.println("    Eso no es un numero.");
                sc.nextLine();
            } else {
                double valor = sc.nextDouble();
                sc.nextLine();
                if (estaEnRango(valor, min, max)) {
                    return valor;
                }
                System.out.println("    Tiene que ir de " + min + " a " + max +
".");
            }
        }
    }

// B11.1
public static boolean leerSiNo(Scanner sc, String pregunta) {
    while (true) {
        String respuesta = leerTexto(sc, pregunta);
        if (respuesta.equalsIgnoreCase("s")) {
            return true;
        }
        if (respuesta.equalsIgnoreCase("n")) {
            return false;
        }
        System.out.println("    Responde s o n.");
    }
}
}

```

B11_1_LeerSiNo.java

```
import java.util.Scanner;

public class B11_1_LeerSiNo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        boolean seguir = Validador.leerSiNo(sc, "Seguir? (s/n): ");
        System.out.println("Respondiste: " + seguir);

        sc.close();
    }
}
```

Salida:

```
--- ejecucion 1
Seguir? (s/n): tal vez
    Responde s o n.
Seguir? (s/n): S
Respondiste: true

--- ejecucion 2
Seguir? (s/n): n
Respondiste: false
```

Lo que importa. `leerSiNo` reutiliza `leerTexto`, así que ya viene con el rechazo de la línea vacía gratis. Construir métodos encima de otros es lo que hace que una clase de utilidades crezca sin volverse un enredo.

Devuelve `boolean`, no `String`. Así el que lo llama escribe `if (Validador.leerSiNo(sc, "Otro? (s/n): "))` sin comparar textos nunca más. Un método bien pensado le ahorra el problema a todos los que lo usan.

El `while (true)` con dos `return` adentro es el mismo patrón de `leerEntero`: se sale por el `return`, y solo con un dato bueno.

B11.2 - La clave segura

Tres condiciones: largo, un dígito y una mayúscula. Las dos últimas obligan a recorrer el texto con un `for`.

▼ Ver la solución (B11_2_ClaveSegura.java)

B11_2_ClaveSegura.java

```
import java.util.Scanner;

public class B11_2_ClaveSegura {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.println(Validador.esClaveSegura("duoc"));           // false
        System.out.println(Validador.esClaveSegura("duocuc2026"));    // false
        System.out.println(Validador.esClaveSegura("DuocUc2026"));    // true
        System.out.println();

        String clave = Validador.leerTexto(sc, "Clave: ");
        while (!Validador.esClaveSegura(clave)) {
            if (clave.length() < 8) {
                System.out.println("    Le faltan caracteres: tiene " +
clave.length() + " de 8.");
            } else {
                System.out.println("    Necesita al menos un numero y una
mayuscula.");
            }
            clave = Validador.leerTexto(sc, "Clave: ");
        }
        System.out.println("Clave aceptada");

        sc.close();
    }
}
```

Salida:

```
false
```

```
false
```

```
true
```

```
Clave: duoc
```

```
    Le faltan caracteres: tiene 4 de 8.
```

```
Clave: duocuc2026
```

```
    Necesita al menos un numero y una mayuscula.
```

```
Clave: DuocUc2026
```

```
Clave aceptada
```

Lo que importa: dos banderas y un solo recorrido. `tieneDigito` y `tieneMayuscula` se van encendiendo mientras el `for` avanza. No se pueden preguntar las dos en el mismo `if`, porque **ningún carácter es a la vez** un dígito y una mayúscula: si escribes `if (isDigit(c) && isUpperCase(c))` la condición nunca se cumple.

El largo se revisa primero y sale de inmediato (`return false`): no tiene sentido recorrer una clave de 3 letras.

Fíjate en que el mensaje de "qué le falta" está en el **programa**, no en el método.

`esClaveSegura` solo responde sí o no; decidir qué decirle al usuario es trabajo de quien la llama. Ese reparto es el que hace que el método sirva también para validar claves que vienen de un archivo, donde no hay a quién avisarle.

B11.3 - El menú blindado

El menú del B6.3 con una sola línea cambiada: la que lee la opción.

▼ Ver la solución (B11_3_MenuBlindado.java)

B11_3_MenuBlindado.java

```
import java.util.Scanner;

public class B11_3_MenuBlindado {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int opcion;

        do {
            System.out.println();
            System.out.println("1) Saludar");
            System.out.println("2) Sumar dos numeros");
            System.out.println("3) Contar hasta 10");
            System.out.println("0) Salir");

            opcion = Validador.leerEntero(sc, "Opcion: ", 0, 3);

            switch (opcion) {
                case 1:
                    System.out.println("Hola!");
                    break;
                case 2:
                    int a = Validador.leerEntero(sc, "Primer numero: ", -1000,
1000);
                    int b = Validador.leerEntero(sc, "Segundo numero: ", -1000,
1000);
                    System.out.println(a + " + " + b + " = " + (a + b));
                    break;
                case 3:
                    for (int i = 1; i <= 10; i++) {
                        System.out.print(i + " ");
                    }
                    System.out.println();
                    break;
                case 0:
                    System.out.println("Chao");
                    break;
                default:
                    System.out.println("Esa opcion no existe");
            }
        } while (opcion != 0);
    }
}
```

```
        sc.close();
    }
}
```

Salida:

```
1) Saludar
2) Sumar dos numeros
3) Contar hasta 10
0) Salir
Opcion: abc
    Eso no es un numero entero.
Opcion: -1
    Tiene que ir de 0 a 3.
Opcion: 99
    Tiene que ir de 0 a 3.
Opcion: 1
Hola!

1) Saludar
2) Sumar dos numeros
3) Contar hasta 10
0) Salir
Opcion: 2
Primer numero: 5
Segundo numero: 7
5 + 7 = 12

1) Saludar
2) Sumar dos numeros
3) Contar hasta 10
0) Salir
Opcion: 0
Chao
```

La respuesta del enunciado: el `default` ya no se puede alcanzar. `leerEntero(sc, ..., 0, 3)` no deja pasar nada fuera de 0 a 3, así que el `default` es código muerto. **Y se deja igual.** Dos razones: si mañana alguien cambia el rango a `0, 5` sin agregar los `case`, el `default` es lo único que evita que el programa quede mudo; y un `switch` sin `default` es una invitación a que el día que falle, falle en silencio.

Fíjate en el `sc.nextLine()` que desapareció. El menú del B6.3 lo necesitaba; este no, porque `leerEntero` se come la línea completa. La trampa del Enter se arregló en un solo lugar, para siempre.

Compara: escribirle `abc` al menú del B6.3 lo mata. A este le rebota.

B11.4 - El RUT

Primero el formato, y después —el extra— el dígito verificador de verdad.

▼ Ver la solución (B11_4_Rut.java)

B11_4_Rut.java

```
public class B11_4_Rut {
    public static void main(String[] args) {
        String[] pruebas = {"12345678-5", "12345678-k", "12345678", "1234a678-5",
                           "12345678-9", "123-4", "12345678--5"};

        System.out.printf("%-14s %-9s %s\n", "rut", "formato", "dv correcto");
        for (String rut : pruebas) {
            System.out.printf("%-14s %-9s %s\n", rut,
                              Validador.esRutValido(rut),
                              Validador.tieneDvCorrecto(rut));
        }
    }
}
```

Salida:

rut	formato	dv correcto
12345678-5	true	true
12345678-k	true	false
12345678	false	false
1234a678-5	false	false
12345678-9	true	false
123-4	false	false
12345678--5	false	false

El formato. `indexOf('-')` devuelve **-1** cuando no hay guion (el mismo -1 del B8.3), así que `guion < 7` rechaza de una vez el RUT sin guion y el que tiene el cuerpo muy corto. Comparar con `lastIndexOf` descarta los que traen dos guiones.

El dígito verificador (módulo 11). Se recorre el cuerpo **de derecha a izquierda** multiplicando por 2, 3, 4, 5, 6, 7 y volviendo al 2. Se suma todo, y el dígito esperado es `11 - (suma % 11)`, con dos casos especiales: 11 es `0` y 10 es `K`.

El detalle que más cuesta: `cuerpo.charAt(i) - '0'` convierte el **carácter** `'7'` en el **número** 7. Restar `'0'` funciona porque los dígitos están seguidos en la tabla de caracteres, y es el mismo truco del `'a' + 1` del B2.1, al revés.

Por eso `12345678-9` tiene formato válido y dígito incorrecto: es exactamente el tipo de dato malo que una validación de formato **no** detecta. En la demo 10, el RUT vuelve a aparecer y ahí el error se avisa lanzando una excepción.

B11.5 - Desafío: la ficha imposible de romper

Siete datos, cada uno con su validación. Los que tienen un `leer...` se resuelven en una línea; los otros, con un `while` y un `es...`.

▼ Ver la solución (B11_5_FichaBlindada.java)

B11_5_FichaBlindada.java

```
import java.util.Scanner;

public class B11_5_FichaBlindada {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        String nombre = Validador.leerTexto(sc, "Nombre: ");

        String rut = Validador.leerTexto(sc, "RUT: ");
        while (!Validador.esRutValido(rut)) {
            System.out.println("    Formato: 12345678-5");
            rut = Validador.leerTexto(sc, "RUT: ");
        }

        int edad = Validador.leerEntero(sc, "Edad: ", 15, 99);

        String correo = Validador.leerTexto(sc, "Correo DUOC: ");
        while (!Validador.esCorreoDuoc(correo)) {
            System.out.println("    Tiene que terminar en @duocuc.cl");
            correo = Validador.leerTexto(sc, "Correo DUOC: ");
        }

        double promedio = Validador.leerDecimal(sc, "Promedio: ", 1.0, 7.0);

        String clave = Validador.leerTexto(sc, "Clave: ");
        while (!Validador.esClaveSegura(clave)) {
            System.out.println("    8 caracteres, un numero y una mayuscula.");
            clave = Validador.leerTexto(sc, "Clave: ");
        }

        String confirmacion = Validador.leerTexto(sc, "Confirmar clave: ");
        while (!confirmacion.equals(clave)) {
            System.out.println("    No coinciden.");
            confirmacion = Validador.leerTexto(sc, "Confirmar clave: ");
        }

        System.out.println();
        System.out.println("===== FICHA =====");
        System.out.printf("%-10s %s\n", "Nombre:", nombre);
        System.out.printf("%-10s %s\n", "RUT:", rut);
        System.out.printf("%-10s %d anios\n", "Edad:", edad);
    }
}
```

```
        System.out.printf("%-10s %s\n", "Correo:", correo);  
        System.out.printf("%-10s %.1f\n", "Promedio:", promedio);  
        System.out.printf("%-10s %s\n", "Clave:", "*****");  
  
        sc.close();  
    }  
}
```

Salida:

```

Nombre:
    No puede quedar vacio.
Nombre:
    No puede quedar vacio.
Nombre: Ana Perez
RUT: 12345678
    Formato: 12345678-5
RUT: 12345678-5
Edad: abc
    Eso no es un numero entero.
Edad: 14
    Tiene que ir de 15 a 99.
Edad: 150
    Tiene que ir de 15 a 99.
Edad: 20
Correo DUOC: ana@gmail.com
    Tiene que terminar en @duocuc.cl
Correo DUOC: ana.perez@duocuc.cl
Promedio: 9.5
    Tiene que ir de 1.0 a 7.0.
Promedio: 0
    Tiene que ir de 1.0 a 7.0.
Promedio: 5.5
Clave: duoc
    8 caracteres, un numero y una mayuscula.
Clave: DuocUc2026
Confirmar clave: otra
    No coinciden.
Confirmar clave: DuocUc2026

===== FICHA =====
Nombre:    Ana Perez
RUT:       12345678-5
Edad:      20 anios
Correo:    ana.perez@duocuc.cl
Promedio:  5.5
Clave:     *****

```

Esa sesión es la tabla de ataques completa del enunciado, y el programa no se cayó ni una vez.

Los dos patrones. Cuando existe un `leer...` para el dato (texto, entero, decimal), es **una línea**. Cuando la regla es propia (RUT, correo, clave), es el patrón `leer + while (!esValido)`. Si te

aparece tres veces el mismo `while`, ya tienes la señal de que ese `while` merece convertirse en un método nuevo del `Validador`.

La clave no se muestra. `printf` imprime `*****` y no la variable: si un programa muestra la clave en pantalla o la guarda tal cual, ya está mal, por más que funcione.

Fíjate en la confirmación: se compara con `equals`, **no** con `==`. Con `==` compararía si son el mismo objeto en memoria y, viniendo las dos del teclado, **nunca** lo son: rechazaría claves idénticas para siempre. Es el paso 5 cobrando su factura.

Volver a la [hoja del paso 11](#) o al [paso 11 de la guía](#).